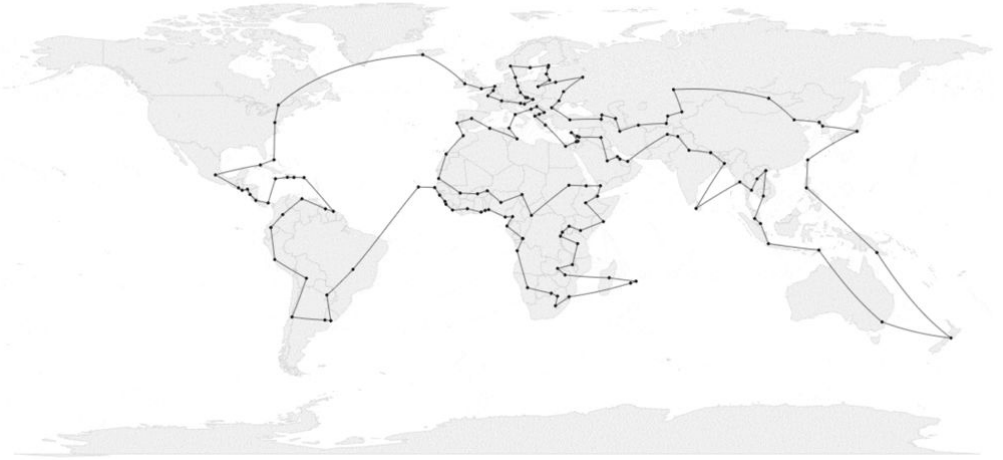


Distance: 80,652 miles
Temperature: 2
Iterations: 1,000,000



Lecture 15

Asymptotics 2

CS61B, Spring 2026 @ UC Berkeley

Josh Hug and Kay Ousterhout

We'll be checking style for the Percolation assignment

- <https://sp26.datastructur.es/resources/style-guide/>

Lecture attendance

- One of the polls will be used to take attendance (do them all! We won't announce which is for attendance)

Asymptotic Review

Lecture 15, CS61B, Spring 2026

More Asymptotic Notation

- **Asymptotic Review**
- Big Omega

Disjoint Sets Review

Asymptotic Analysis Examples

- Nested For Loops
- A Trickier Nested For Loop
- There is No Magic Shortcut for Asymptotic Analysis
- More Loop Examples

Summary of Previous Asymptotics Lecture

Given a code snippet, we can express its runtime as a function $R(N)$, where N is some property of the input of the function (often the size of the input).

Rather than finding $R(N)$ exactly, we instead usually only care about the order of growth of $R(N)$.

println could be our
representative operation

```
public static void printParty(int N) {  
    for (int i = 10; i < N; i += 2) {  
        System.out.println("hello");  
    }  
}
```

One approach (not universal):

- Choose a representative operation (**cost model**), and let $C(N)$ be the count of how many times that operation occurs as a function of N .
- Determine order of growth $f(N)$ for $C(N)$, i.e. $C(N) \in \Theta(f(N))$
 - Often (but not always) we consider the worst case count.
- If operation (e.g., `println`) takes constant time, then $R(N) \in \Theta(f(N))$



What's the order of growth of printParty?

println could be our
representative operation

```
public static void printParty(int N) {  
    for (int i = 10; i < N; i += 2) {  
        System.out.println("hello");  
    }  
}
```

One approach (not universal):

- Choose a representative operation (**cost model**), and let $C(N)$ be the count of how many times that operation occurs as a function of N .
- Determine order of growth $f(N)$ for $C(N)$, i.e. $C(N) \in \Theta(f(N))$
 - Often (but not always) we consider the worst case count.
- If operation (e.g., `println`) takes constant time, then $R(N) \in \Theta(f(N))$

What's the order of growth of printParty?

How many println calls? $(N - 10) / 2$

Runtime is $\Theta(N)$

Simplifications:

- Ignore low orders terms
- Ignore multiplicative constants

println could be our
representative operation

```
public static void printParty(int N) {  
    for (int i = 10; i < N; i += 2) {  
        System.out.println("hello");  
    }  
}
```

One approach (not universal):

- Choose a representative operation (**cost model**), and let $C(N)$ be the count of how many times that operation occurs as a function of N .
- Determine order of growth $f(N)$ for $C(N)$, i.e. $C(N) \in \Theta(f(N))$
 - Often (but not always) we consider the worst case count.
- If operation (e.g., println) takes constant time, then $R(N) \in \Theta(f(N))$

Summary of Previous Asymptotics Lecture

What's the order of growth of printParty?

How many println calls? $(N - 10) / 2$

Runtime is $\Theta(N)$

Simplifications:

- Ignore low orders terms
- Ignore multiplicative constants

println could be our
representative operation

```
public static void printParty(int N) {  
    for (int i = 10; i < N; i += 2) {  
        System.out.println("hello");  
    }  
}
```

Why'd we drop the -10?

Why'd we drop the / 2?

What's the order of growth of printParty?

How many println calls? $(N - 10) / 2$

Runtime is $\Theta(N)$

Simplifications:

- Ignore low orders terms
- Ignore multiplicative constants

println could be our
representative operation

```
public static void printParty(int N) {  
    for (int i = 10; i < N; i += 2) {  
        System.out.println("hello");  
    }  
}
```

Why'd we drop the -10?

- Becomes insignificant as N gets big

Why'd we drop the / 2?

- We're trying to describe how the runtime grows with N
- We've already dropped a bunch of constant factors, e.g., how long does println take to run? How fast is my laptop?

Big Theta is a mathematical tool to describe the order of growth of a function.

function $R(N)$	order of growth
$N^3 + 3N^4$	$\Theta(N^4)$
$1/N + N^3$	$\Theta(N^3)$
$1/N + 5$	$\Theta(1)$
$N^e + N$	$\Theta(N^e)$
$40 \sin(N) + 4N^2$	$\Theta(N^2)$

We also used Big Theta to describe the rate of growth of the runtime of a piece of code.

- Idea was simple: Assume runtime is a function of the input.
- Example: Assume runtime of `printParty` is proportional to # of prints.

Whereas Big Theta can informally be thought of as something like “equals”, Big O can be thought of as “less than or equal”.

Example, the following are all true:

- $N^3 + 3N^4 \in \Theta(N^4)$
- $N^3 + 3N^4 \in O(N^4)$
- $N^3 + 3N^4 \in O(N^6)$
- $N^3 + 3N^4 \in O(N!)$
- $N^3 + 3N^4 \in O(N^{N!})$

$$R(N) \in \Theta(f(N))$$

means there exist positive constants k_1 and k_2 such that:

$$k_1 \cdot f(N) \leq R(N) \leq k_2 \cdot f(N)$$

for all values of N greater than some N_0 .  i.e. very large N

$$R(N) \in O(f(N))$$

means there exist positive constants k_1 and k_2 such that:

$$R(N) \leq k_2 \cdot f(N)$$

for all values of N greater than some N_0 .  i.e. very large N

Big O is just an upper bound!

Big Omega

Lecture 15, CS61B, Spring 2026

More Asymptotic Notation

- Asymptotic Review
- **Big Omega**

Disjoint Sets Review

Asymptotic Analysis Examples

- Nested For Loops
- A Trickier Nested For Loop
- There is No Magic Shortcut for Asymptotic Analysis
- More Loop Examples

Whereas Big O is an upper bound, Big Omega is a lower bound

Example, the following are all true:

- $N^3 + 3N^4 \in \Theta(N^4)$
- $N^3 + 3N^4 \in \Omega(N^4)$
- $N^3 + 3N^4 \in \Omega(N^3)$
- $N^3 + 3N^4 \in \Omega(\log N)$
- $N^3 + 3N^4 \in \Omega(1)$

$$R(N) \in \Omega(f(N))$$

means there exists a positive constant k_1 such that:

$$k_1 \cdot f(N) \leq R(N)$$

for all values of N greater than some N_0 .

← i.e. very large N

Example: $40 \sin(N) + 4N^2 \in \Omega(N)$

- $R(N) = 40 \sin(N) + 4N^2$
- $f(N) = N$
- $k_1 = 20$

Big Omega is often used for setting theoretical floors on problem difficulty.

For example, finding the median of a list of numbers takes $\Omega(N)$.

- That is, among the space of ALL possible algorithms for finding the median, the best you could ever do is linear time. Why?

We'll use Ω in a much later lecture.

Big Theta, Big O, and Big Omega

	Informal meaning:	Family	Family Members
Big Theta $\Theta(f(N))$	Order of growth is $f(N)$.	$\Theta(N^2)$	$N^2/2$ $2N^2$ $N^2 + 38N + N$
Big O $O(f(N))$	Order of growth is less than or equal to $f(N)$.	$O(N^2)$	$N^2/2$ $2N^2$ $\lg(N)$
Big Omega $\Omega(f(N))$	Order of growth is greater than or equal to $f(N)$.	$\Omega(N^2)$	$N^2/2$ $2N^2$ e^N

Disjoint Sets Review

Lecture 15, CS61B, Spring 2026

More Asymptotic Notation

- Asymptotic Review
- Big Omega

Disjoint Sets Review

Asymptotic Analysis Examples

- Nested For Loops
- A Trickier Nested For Loop
- There is No Magic Shortcut for Asymptotic Analysis
- More Loop Examples

Disjoint Sets Performance Summary

Implementation	constructor	connect	isConnected
ListOfSetsDS	$\Theta(N)$ Make N sets	$O(N)$ Find each element, Combine 2 sets	$O(N)$ Check up to N sets
QuickFindDS id array holds set ID for each item	$\Theta(N)$ Initialize size-N array	$\Theta(N)$ Look through entire array, update IDs	$\Theta(1)$ Two array accesses
QuickUnionDS parent array holds each item's parent in tree structure	$\Theta(N)$ Initialize size-N array	$O(N)$ Climb tree to find root	$O(N)$ Climb 2 tree to compare roots
WeightedQuickUnionDS Like QuickUnionDS, but track tree size; connect smaller tree under bigger tree	$\Theta(N)$ Initialize size-N array	$O(\log N)$ Climb tree to find root	$O(\log N)$ Climb 2 tree to compare roots

Why are some entries Big O instead of Big Omega (Θ)?

Implementation	constructor	connect	isConnected
ListOfSetsDS	$\Theta(N)$ Make N sets	$O(N)$ Find each element, Combine 2 sets	$O(N)$ Check up to N sets
QuickFindDS id array holds set ID for each item	$\Theta(N)$ Initialize size-N array	$\Theta(N)$ Look through entire array, update IDs	$\Theta(1)$ Two array accesses
QuickUnionDS parent array holds each item's parent in tree structure	$\Theta(N)$ Initialize size-N array	$O(N)$ Climb tree to find root	$O(N)$ Climb 2 tree to compare roots
WeightedQuickUnionDS Like QuickUnionDS, but track tree size; connect smaller tree under bigger tree	$\Theta(N)$ Initialize size-N array	$O(\log N)$ Climb tree to find root	$O(\log N)$ Climb 2 tree to compare roots

Why are some entries Big O instead of Big Omega (Θ)?

Implementation	constructor	connect	isConnected
ListOfSetsDS	$\Theta(N)$ Make N sets	$O(N)$ Find each element, Combine 2 sets	$O(N)$ Check up to N sets
QuickFindDS id array holds set ID for each item	$\Theta(N)$ Initialize size-N array	$\Theta(N)$ Look through entire array, update IDs	$\Theta(1)$ Two array accesses
QuickUnionDS parent array holds each item's parent in tree structure	$\Theta(N)$ Initialize size-N array	$O(N)$ Climb tree to find root	$O(N)$ Climb 2 tree to compare roots
WeightedQuickUnionDS Like QuickUnionDS, but track tree size; connect smaller tree under bigger tree	$\Theta(N)$ Initialize size-N array	$O(\log N)$ Climb tree to find root	$O(\log N)$ Climb 2 tree to compare roots

**Might be much faster, e.g.,
constant time**

Nested For Loops

Lecture 15, CS61B, Spring 2026

More Asymptotic Notation

- Asymptotic Review
- Big Omega

Disjoint Sets Review

Asymptotic Analysis Examples

- **Nested For Loops**
- A Trickier Nested For Loop
- There is No Magic Shortcut for Asymptotic Analysis
- More Loop Examples

Loops Example 1: Counting Duplicates

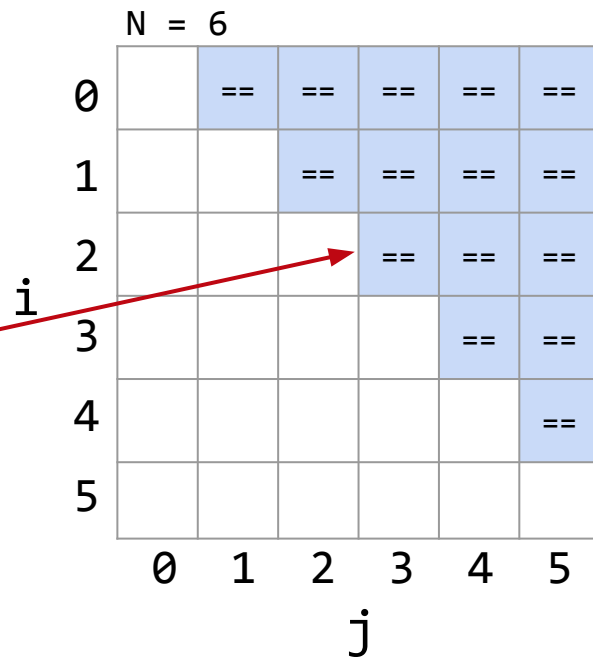
```
public static void countDuplicates(int[] a) {  
    int duplicates = 0;  
    for (int i = 0; i < a.length; i++) {  
        for (int j = i + 1; j < a.length; j++) {  
            if (a[i] == a[j]) {  
                duplicates += 1;  
            }  
        }  
    }  
    IO.println("Duplicates: " + duplicates);  
}
```

Intuitive approach: how many pairs do we check?

Cost model: ==

What's the order growth of countDuplicates?

- Area of right triangle: $\sim N^2/2$
- **Order of growth is N^2**

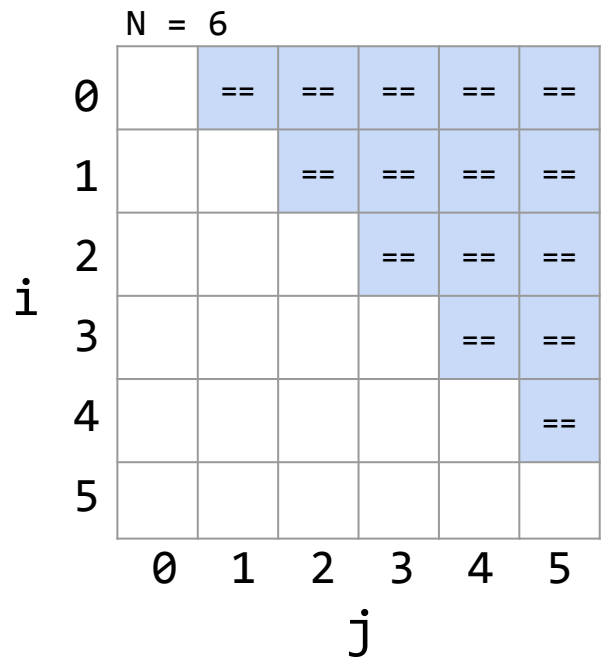
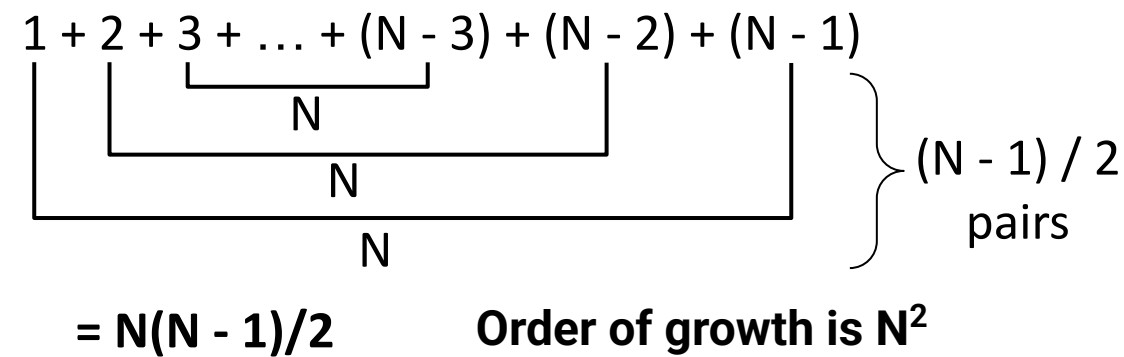


Loops Example 1: Counting Duplicates

```
public static void countDuplicates(int[] a) {
    int duplicates = 0;
    for (int i = 0; i < a.length; i++) {
        for (int j = i + 1; j < a.length; j++) {
            if (a[i] == a[j]) {
                duplicates += 1;
            }
        }
    }
    IO.println("Duplicates: " + duplicates);
}
```

Intuitive approach: how many pairs do we check?

Cost model: ==



Let $R(N)$ be the runtime of `scan`. Find a simple $f(N)$ such that $R(N) \in \Theta(f(N))$. By simple, we mean there should be no unnecessary multiplicative constants or additive terms.



```
public static void scan(int N) {  
    int s = 0;  
    for (int i = 2; i <= N; i += 1) {  
        for (int j = i-1; j <= i+1; j += 1) {  
            s += i*j;  
        }  
    }  
}
```

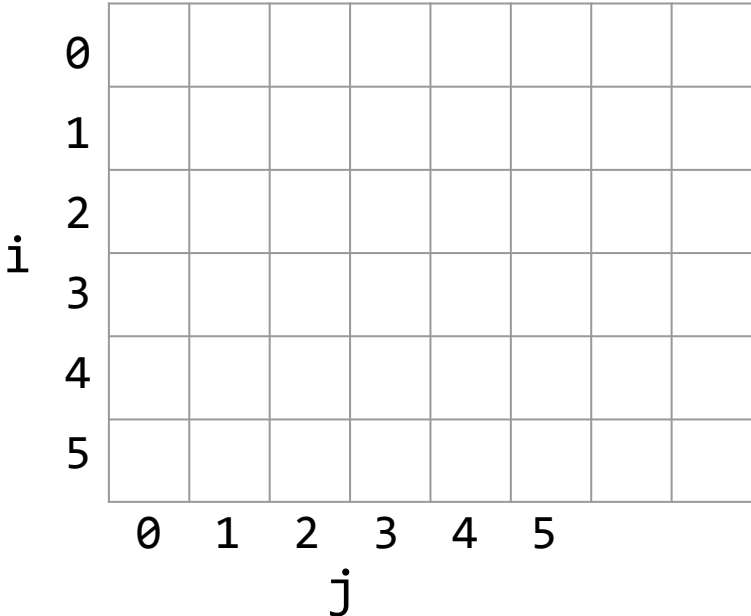
- A. 1
- B. $\log N$
- C. N
- D. $N \log N$
- E. N^2
- F. Other

Loops Example 2: Arithmetic Argument

Find the order of growth of the worst case runtime of scan.

```
public static void scan(int N) {  
    int s = 0;  
    for (int i = 2; i <= N; i += 1) {  
        for (int j = i-1; j <= i+1; j += 1) {  
            s += i*j;  
        }  
    }  
}
```

N	s += i*j calls
0	0
1	0
2	
3	
4	
5	

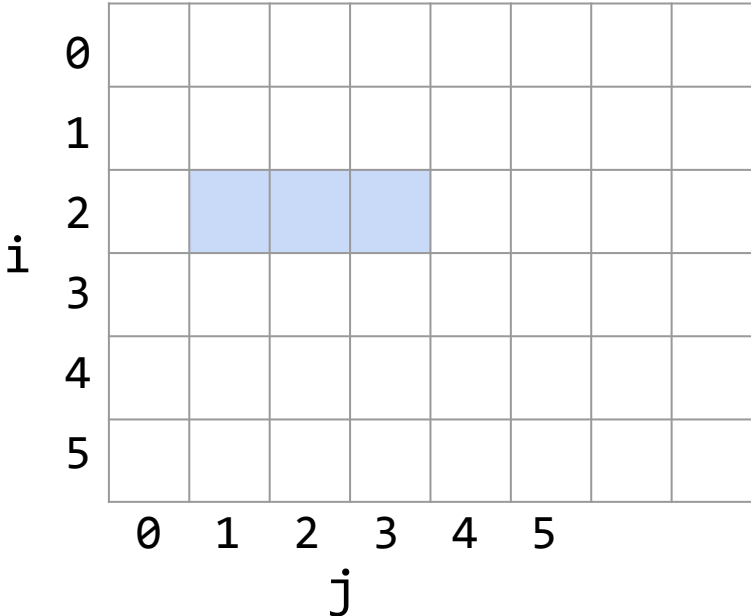


Loops Example 2: Arithmetic Argument

Find the order of growth of the worst case runtime of scan.

```
public static void scan(int N) {  
    int s = 0;  
    for (int i = 2; i <= N; i += 1) {  
        for (int j = i-1; j <= i+1; j += 1) {  
            s += i*j;  
        }  
    }  
}
```

N	s += i*j calls
0	0
1	0
2	3
3	
4	
5	

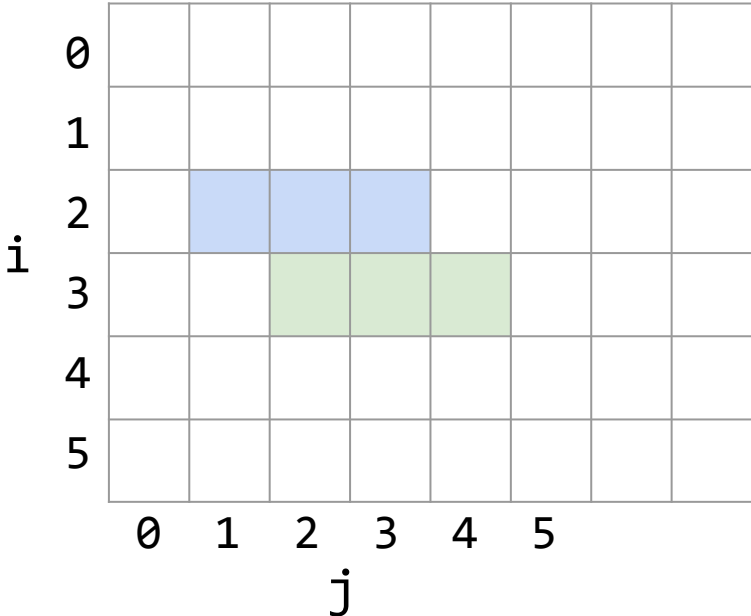


Loops Example 2: Arithmetic Argument

Find the order of growth of the worst case runtime of scan.

```
public static void scan(int N) {  
    int s = 0;  
    for (int i = 2; i <= N; i += 1) {  
        for (int j = i-1; j <= i+1; j += 1) {  
            s += i*j;  
        }  
    }  
}
```

N	s += i*j calls
0	0
1	0
2	3
3	?
4	
5	

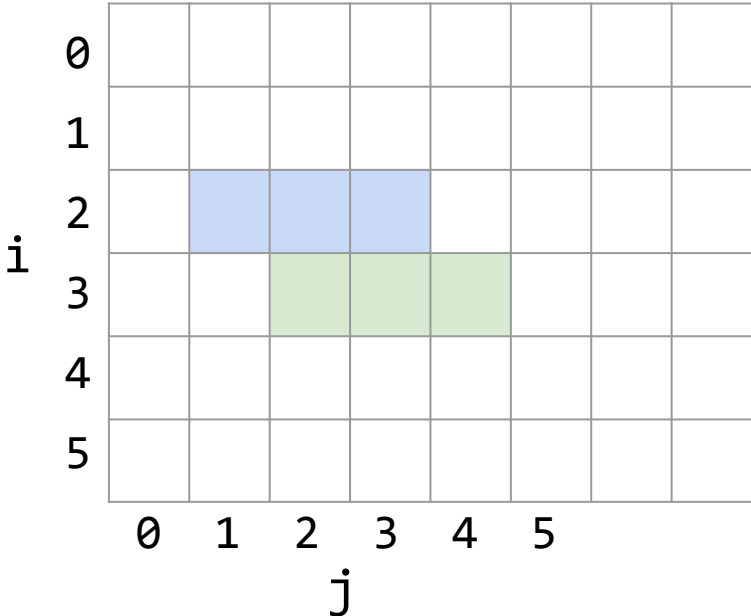


Loops Example 2: Arithmetic Argument

Find the order of growth of the worst case runtime of scan.

```
public static void scan(int N) {  
    int s = 0;  
    for (int i = 2; i <= N; i += 1) {  
        for (int j = i-1; j <= i+1; j += 1) {  
            s += i*j;  
        }  
    }  
}
```

N	s += i*j calls
0	0
1	0
2	3
3	3+3=6
4	
5	



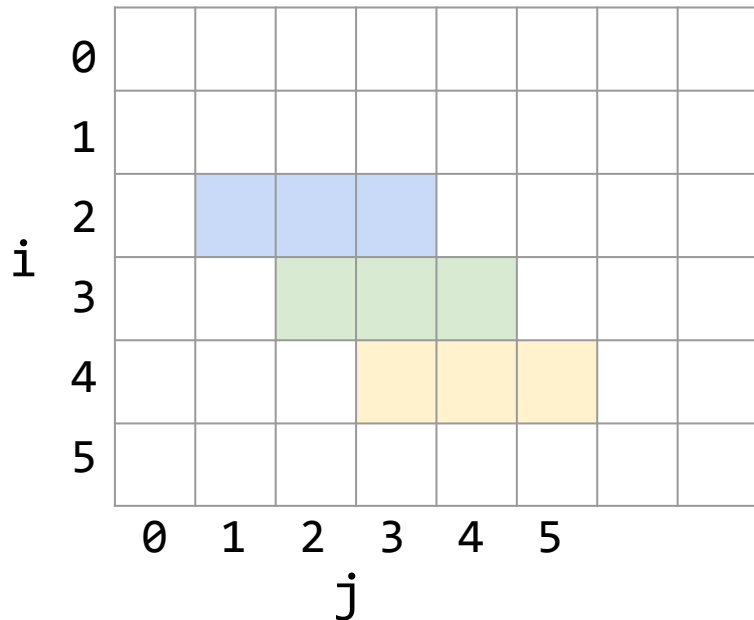
Loops Example 2: Arithmetic Argument

Find the order of growth of the worst case runtime of scan.

```
public static void scan(int N) {  
    int s = 0;  
    for (int i = 2; i <= N; i += 1) {  
        for (int j = i-1; j <= i+1; j += 1) {  
            s += i*j;  
        }  
    }  
}
```

N s += i*j calls

0	0
1	0
2	3
3	3+3=6
4	3+3+3=9
5	



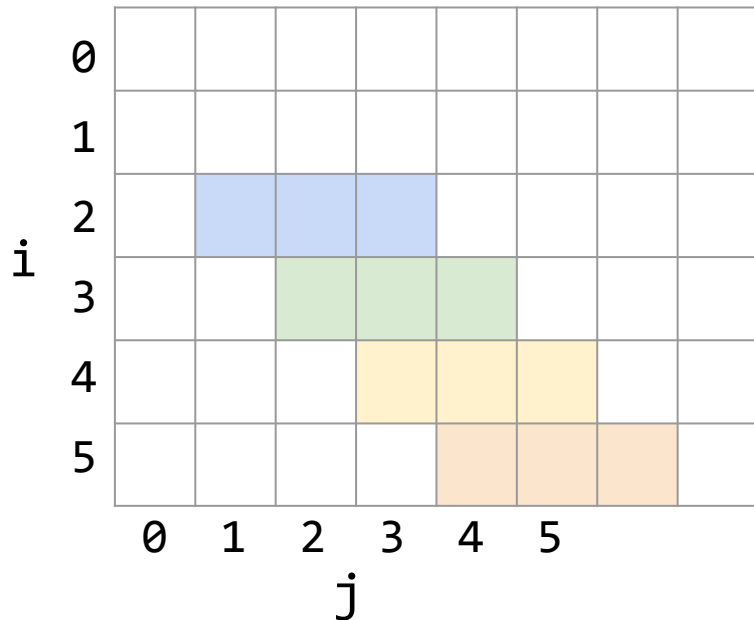
Loops Example 2: Arithmetic Argument

Find the order of growth of the worst case runtime of scan.

```
public static void scan(int N) {  
    int s = 0;  
    for (int i = 2; i <= N; i += 1) {  
        for (int j = i-1; j <= i+1; j += 1) {  
            s += i*j;  
        }  
    }  
}
```

N s += i*j calls

0	0
1	0
2	3
3	3+3=6
4	3+3+3=9
5	12

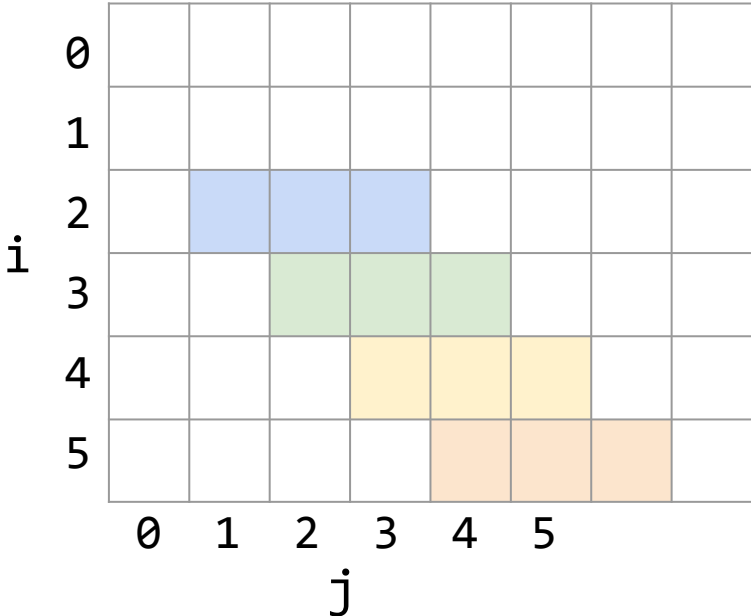


Loops Example 2: Arithmetic Argument

Find the order of growth of the worst case runtime of scan.

```
public static void scan(int N) {
    int s = 0;
    for (int i = 2; i <= N; i += 1) {
        for (int j = i-1; j <= i+1; j += 1) {
            s += i*j;
        }
    }
}
```

N	s += i*j calls
0	0
1	0
2	3
3	3+3=6
4	3+3+3=9
5	12



- Worst case number of $s += i*j$ operations:
- Given by area of line of width 3 and length $N - 1 = 3 * (N-1)$
 - Overall area is $\Theta(N)$

Loops Example 2: Arithmetic Argument

Find the order of growth of the worst case runtime of scan.

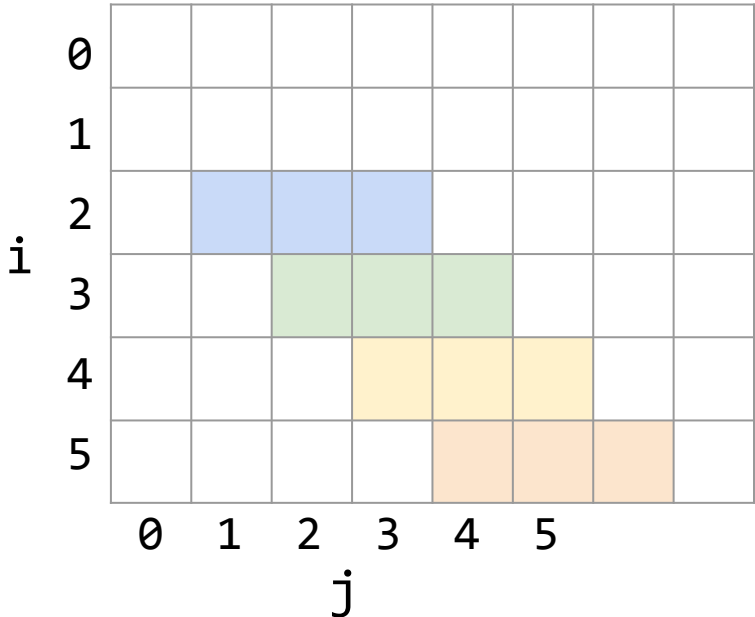
$R(N) \in \Theta(N)$ because $s += i*j$ was a good cost model

```
public static void scan(int N) {
    int s = 0;
    for (int i = 2; i <= N; i += 1) {
        for (int j = i-1; j <= i+1; j += 1) {
            s += i*j;
        }
    }
}
```

Counting $i = 2$ would have been a bad cost model

N s += i*j calls

0	0
1	0
2	3
3	3+3=6
4	3+3+3=9
5	12



- Worst case number of $s += i*j$ operations:
- Given by area of line of width 3 and length $N - 1 = 3 * (N-1)$
 - Overall area is $\Theta(N)$

A Trickier Nested For Loop

Lecture 15, CS61B, Spring 2026

More Asymptotic Notation

- Asymptotic Review
- Big Omega

Disjoint Sets Review

Asymptotic Analysis Examples

- Nested For Loops
- A Trickier Nested For Loop
- There is No Magic Shortcut for Asymptotic Analysis
- More Loop Examples

Find a simple $f(N)$ such that the runtime $R(N) \in \Theta(f(N))$. By simple, we mean there should be no unnecessary multiplicative constants or additive terms.

```
public static void printParty(int N) {  
    for (int i = 1; i <= N; i = i * 2) {  
        for (int j = 0; j < i; j += 1) {  
            System.out.println("hello");  
            int ZUG = 1 + 1;  
        }  
    }  
}
```



- | | |
|-------------|---------------|
| A. 1 | D. $N \log N$ |
| B. $\log N$ | E. N^2 |
| C. N | F. Other |

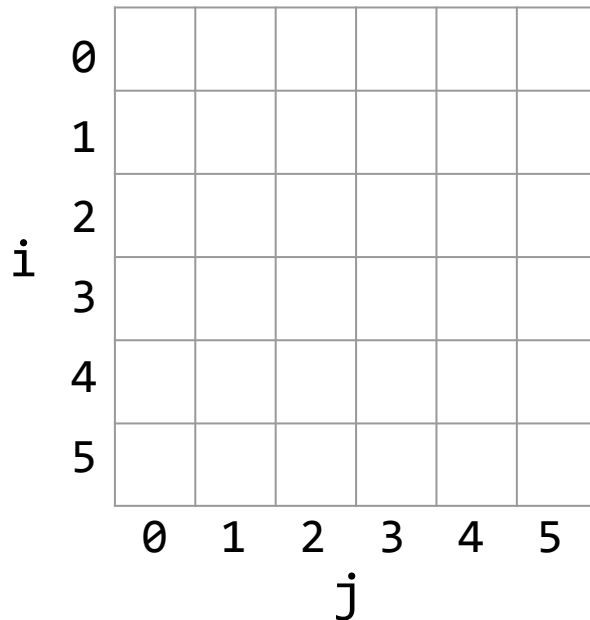
Loops Example 3: Prelude to Attempt #2

Find a simple $f(N)$ such that the runtime $R(N) \in \Theta(f(N))$.

N println calls

0	0
1	
2	
3	
4	
5	

```
public static void printParty(int N) {  
    for (int i = 1; i <= N; i = i * 2) {  
        for (int j = 0; j < i; j += 1) {  
            System.out.println("hello");  
            int ZUG = 1 + 1;  
        }  
    }  
}
```



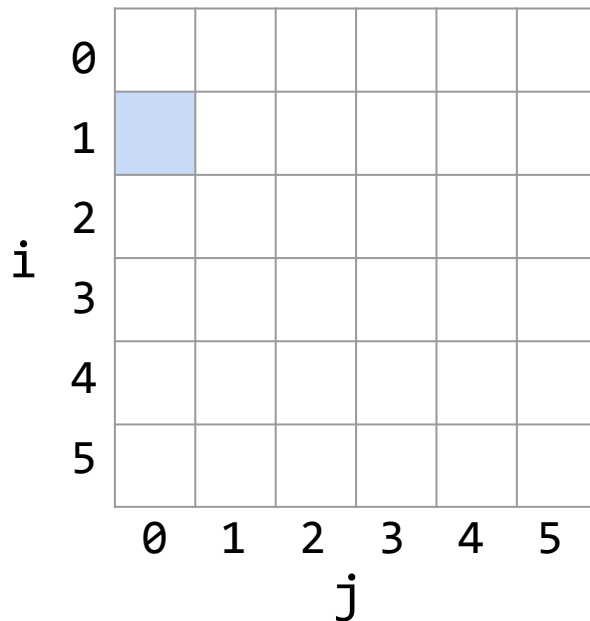
Loops Example 3: Prelude to Attempt #2

Find a simple $f(N)$ such that the runtime $R(N) \in \Theta(f(N))$.

N println calls

0	0
1	1
2	
3	
4	
5	

```
public static void printParty(int N) {  
    for (int i = 1; i <= N; i = i * 2) {  
        for (int j = 0; j < i; j += 1) {  
            System.out.println("hello");  
            int ZUG = 1 + 1;  
        }  
    }  
}
```



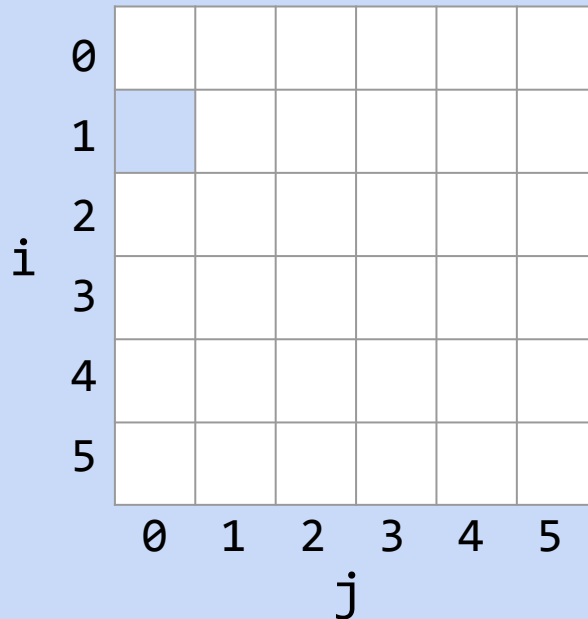
Loops Example 3: Prelude to Attempt #2

Find a simple $f(N)$ such that the runtime $R(N) \in \Theta(f(N))$.

N println calls

0	0
1	1
2	?
3	?
4	?
5	?

```
public static void printParty(int N) {  
    for (int i = 1; i <= N; i = i * 2) {  
        for (int j = 0; j < i; j += 1) {  
            System.out.println("hello");  
            int ZUG = 1 + 1;  
        }  
    }  
}
```



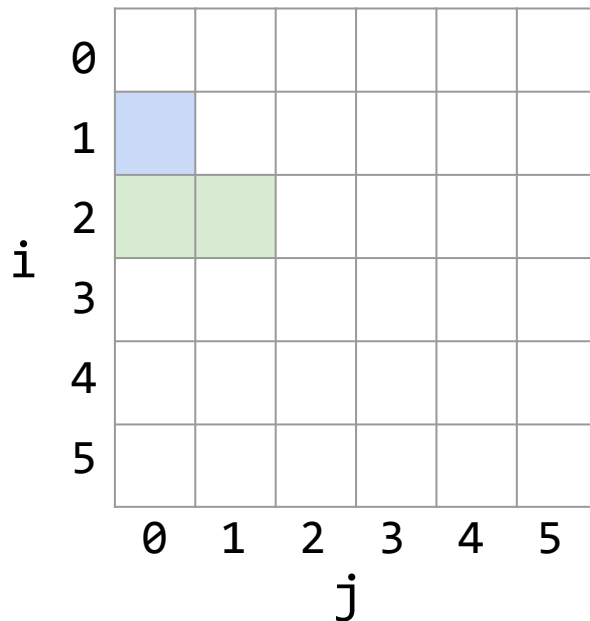
Loops Example 3: Prelude to Attempt #2

Find a simple $f(N)$ such that the runtime $R(N) \in \Theta(f(N))$.

N println calls

0	0
1	1
2	1+2=3
3	?
4	?
5	?

```
public static void printParty(int N) {  
    for (int i = 1; i <= N; i = i * 2) {  
        for (int j = 0; j < i; j += 1) {  
            System.out.println("hello");  
            int ZUG = 1 + 1;  
        }  
    }  
}
```



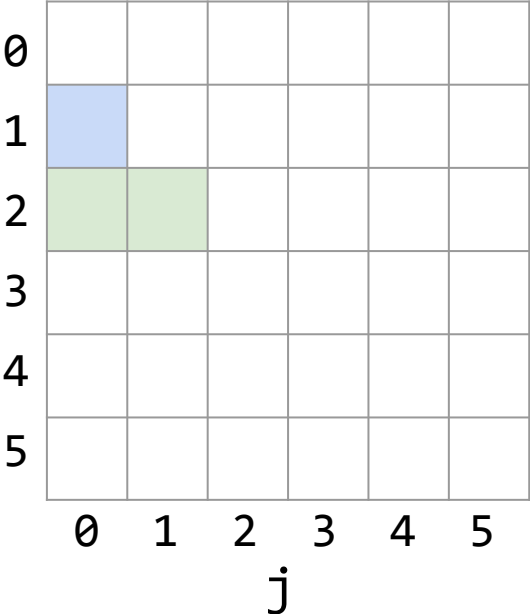
Loops Example 3: Prelude to Attempt #2

Find a simple $f(N)$ such that the runtime $R(N) \in \Theta(f(N))$.

```
public static void printParty(int N) {
    for (int i = 1; i <= N; i = i * 2) {
        for (int j = 0; j < i; j += 1) {
            System.out.println("hello");
            int ZUG = 1 + 1;
        }}}
```

N println calls

0	0	N=2 and 3 both print 3 times
1	1	
2	1+2=3	
3	3	
4	?	
5	?	



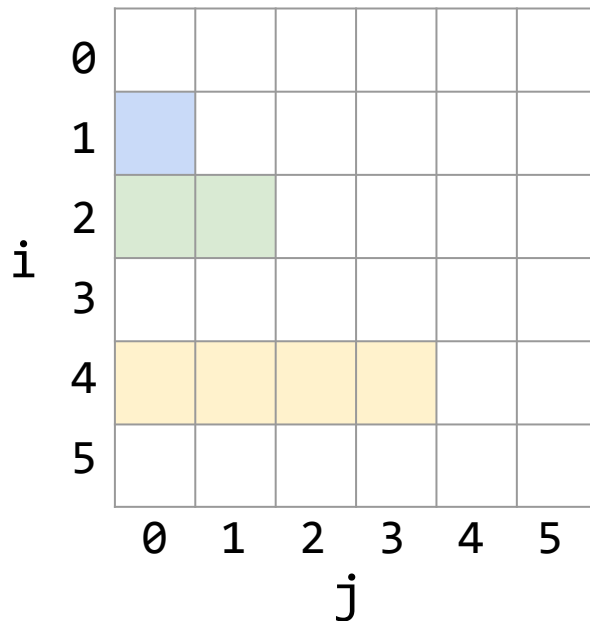
Loops Example 3: Prelude to Attempt #2

Find a simple $f(N)$ such that the runtime $R(N) \in \Theta(f(N))$.

N println calls

0	0
1	1
2	1+2=3
3	3
4	1+2+4=7
5	?

```
public static void printParty(int N) {  
    for (int i = 1; i <= N; i = i * 2) {  
        for (int j = 0; j < i; j += 1) {  
            System.out.println("hello");  
            int ZUG = 1 + 1;  
        }  
    }  
}
```



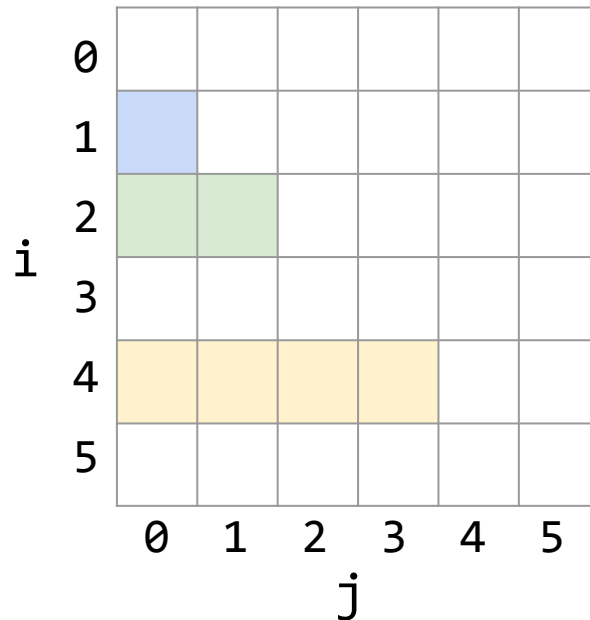
Loops Example 3: Prelude to Attempt #2

Find a simple $f(N)$ such that the runtime $R(N) \in \Theta(f(N))$.

```
public static void printParty(int N) {  
    for (int i = 1; i <= N; i = i * 2) {  
        for (int j = 0; j < i; j += 1) {  
            System.out.println("hello");  
            int ZUG = 1 + 1;  
        }  
    }  
}
```

N println calls

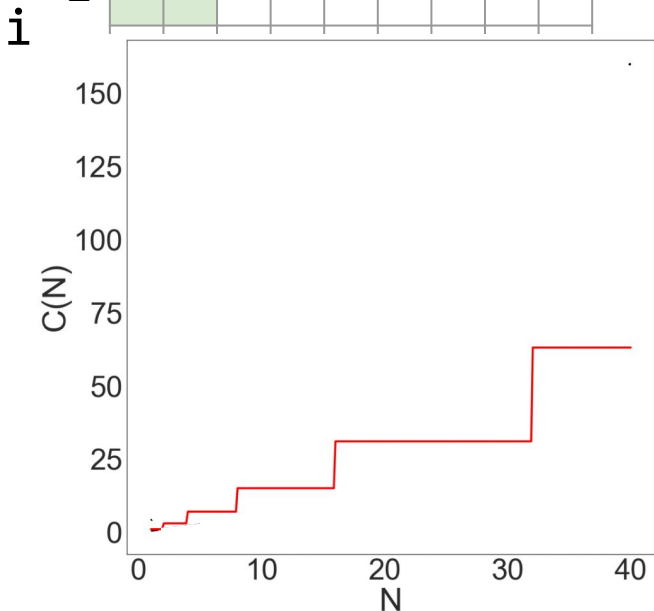
0	0
1	1
2	1+2=3
3	3
4	1+2+4=7
5	7



Loops Example 3: Prelude to Attempt #2

N	println calls	j								
		0	1	2	3	4	5	6	7	8
0	0									
1	1									
2	1+2=3									
3	3									
4	1+2+4=7									
5	7									
6	7									
7	7									
8	1+2+4+8=15									
9	15									

```
public static void printParty(int N) {
    for (int i = 1; i <= N; i = i * 2) {
        for (int j = 0; j < i; j += 1) {
            System.out.println("hello");
            int ZUG = 1 + 1;
        }
    }
}
```



println calls (C(N)):

If N is a power of 2:
 $C(N) = 1 + 2 + 4 + \dots + N$

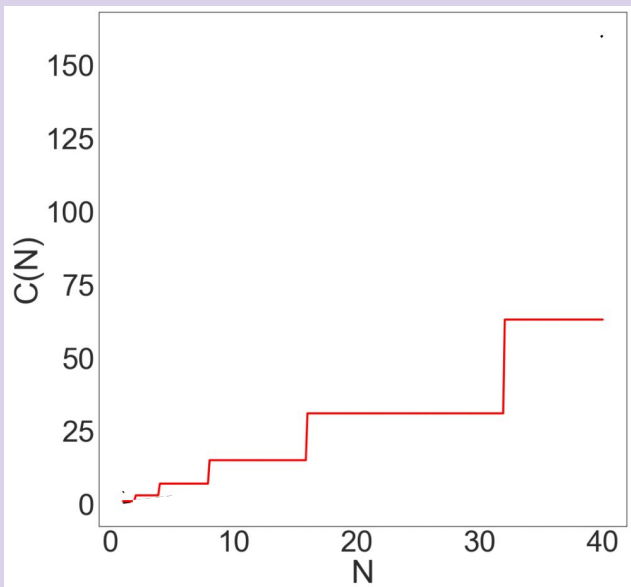
Example: N = 8

- $C(N) = 1 + 2 + 4 + 8 = 15$

Find a simple $f(N)$ such that the runtime $R(N) \in \Theta(f(N))$.

- | | |
|-------------|---------------|
| A. 1 | D. $N \log N$ |
| B. $\log N$ | E. N^2 |
| C. N | F. Other |

```
public static void printParty(int N) {  
    for (int i = 1; i <= N; i = i * 2) {  
        for (int j = 0; j < i; j += 1) {  
            System.out.println("hello");  
            int ZUG = 1 + 1;  
        }  
    }  
}
```



println calls ($C(N)$):

If N is a power of 2:

$$C(N) = 1 + 2 + 4 + \dots + N$$

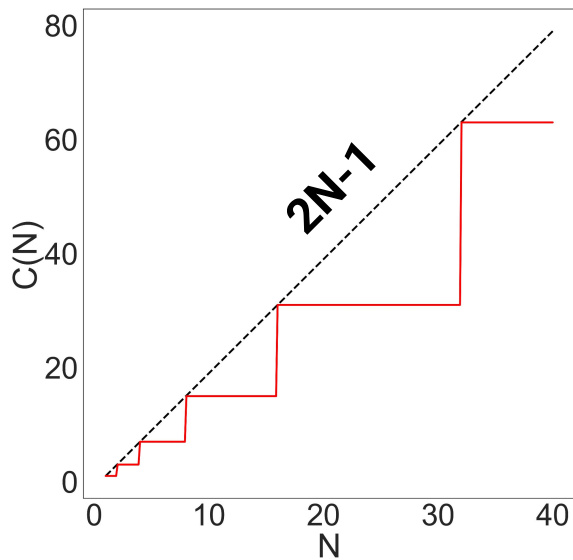
Example: $N = 8$

- $C(N) = 1 + 2 + 4 + 8 = 15$

Loops Example 3: Prelude to Attempt #3

N println calls

0	0
1	1
2	1+2=3
3	3
4	1+2+4=7
5	7
6	7
7	7
8	1+2+4+8=15
9	16



```
public static void printParty(int N) {  
    for (int i = 1; i <= N; i = i * 2) {  
        for (int j = 0; j < i; j += 1) {  
            System.out.println("hello");  
            int ZUG = 1 + 1;  
        }  
    }  
}
```

println calls (C(N)):

If N is a power of 2:

$$C(N) = 1 + 2 + 4 + \dots + N$$

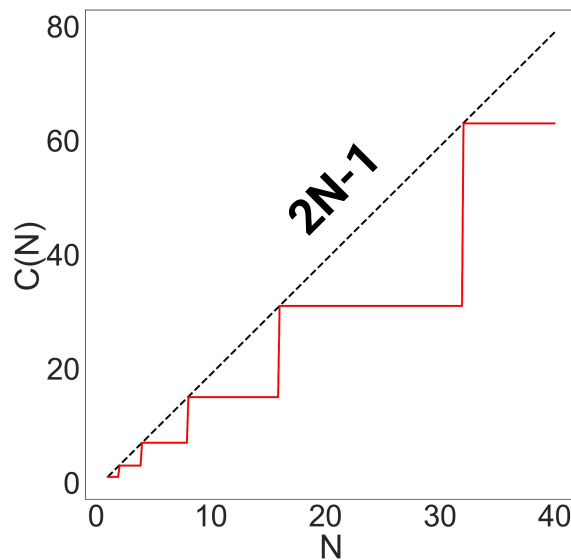
Example: N = 8

- $C(N) = 1 + 2 + 4 + 8 = 15$

Loops Example 3: Prelude to Attempt #3

N println calls

0	0
1	1
2	1+2=3
3	3
4	1+2+4=7
5	7
6	7
7	7
8	1+2+4+8=15
9	16



```
public static void printParty(int N) {  
    for (int i = 1; i <= N; i = i * 2) {  
        for (int j = 0; j < i; j += 1) {  
            System.out.println("hello");  
            int ZUG = 1 + 1;  
        }  
    }  
}
```

println calls (C(N)):

If N is a power of 2:

$$C(N) = 1 + 2 + 4 + \dots + N = 2N - 1$$

Example: N = 8

- $C(N) = 1 + 2 + 4 + 8 = 15$

Proof that $1 + 2 + 4 + \dots + N = 2N - 1$

Claim: If N is a power of 2, then $1 + 2 + 4 + \dots + N = 2N - 1$

Proof by induction:

- Base case, let $N = 1$.
 - $2 \cdot 1 - 1 = 1$

Inductive hypothesis: Sum of powers of 2 up to $N = 1 + 2 + 4 + \dots + N = 2N - 1$

- After N , the next power of two is $2N$.
- Thus, we want to show $1 + 2 + 4 + \dots + N + 2N = 4N - 1$
- We start by assuming the inductive hypothesis: $1 + 2 + 4 + \dots + N = 2N - 1$
 - Then we add $2N$ to both sides.
 - This gives us that $1 + 2 + 4 + \dots + N + 2N = 2N - 1 + 2N$.

$$= 4N - 1$$

You might ask: Why are you writing this so weirdly? You wrote:

- Suppose N is a power of 2, then $1 + 2 + 4 + \dots + N = 2N - 1$.

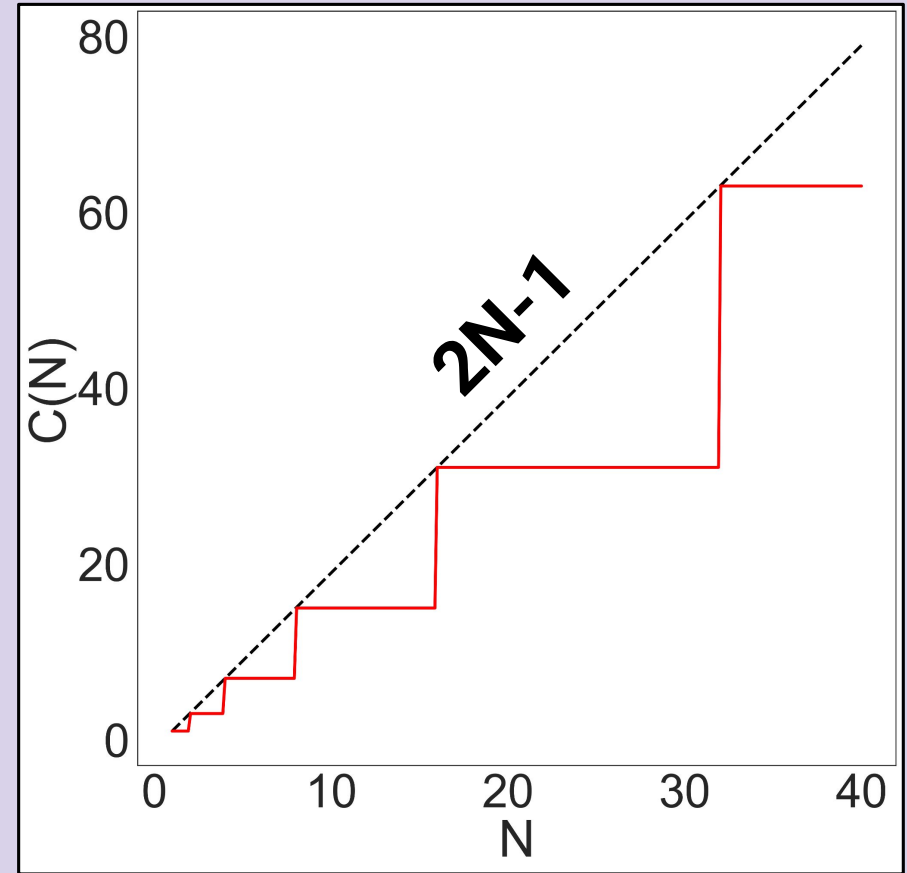
But it seems more natural to write this as: Suppose N is a power of 2, i.e. $N = 2^k$ for some k , then:

$$1 + 2 + 2^2 + \dots + 2^k = \sum_{i=0}^k 2^i = 2^{k+1} - 1$$

Experience has shown me that 61B students get k and N confused and this more formal framing ends up being a distraction.

Find a simple $f(N)$ such that $C(N) \in \Theta(f(N))$.

- | | |
|-------------|---------------|
| A. 1 | D. $N \log N$ |
| B. $\log N$ | E. N^2 |
| C. N | F. Other |



Solution: $C(N)$ is Linear

Find a simple $f(N)$ such that $C(N) \in \Theta(f(N))$.

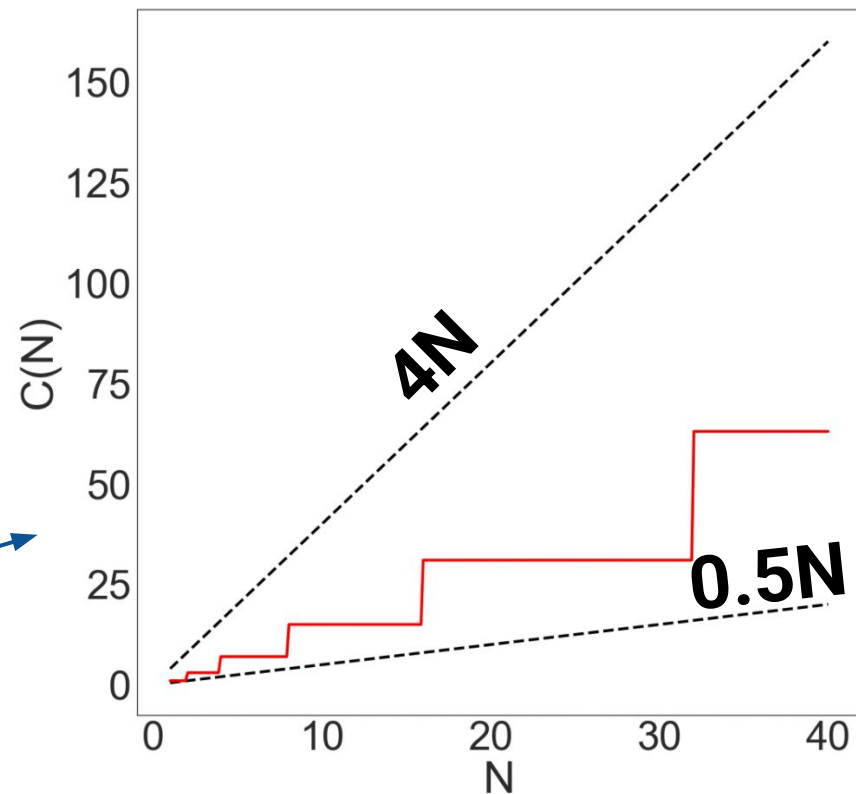
- | | |
|--------------------------|---------------|
| A. 1 | D. $N \log N$ |
| B. $\log N$ | E. N^2 |
| C. N | F. Other |

Recall that $C(N) \in \Theta(N)$ means that we can find k_1 and k_2 such that for large N :

- $k_1 * N \leq C(N) \leq k_2 * N$

For example:

- $k_1 = 0.5, k_2 = 4$
- $k_1 = 0.001, k_2 = 103$



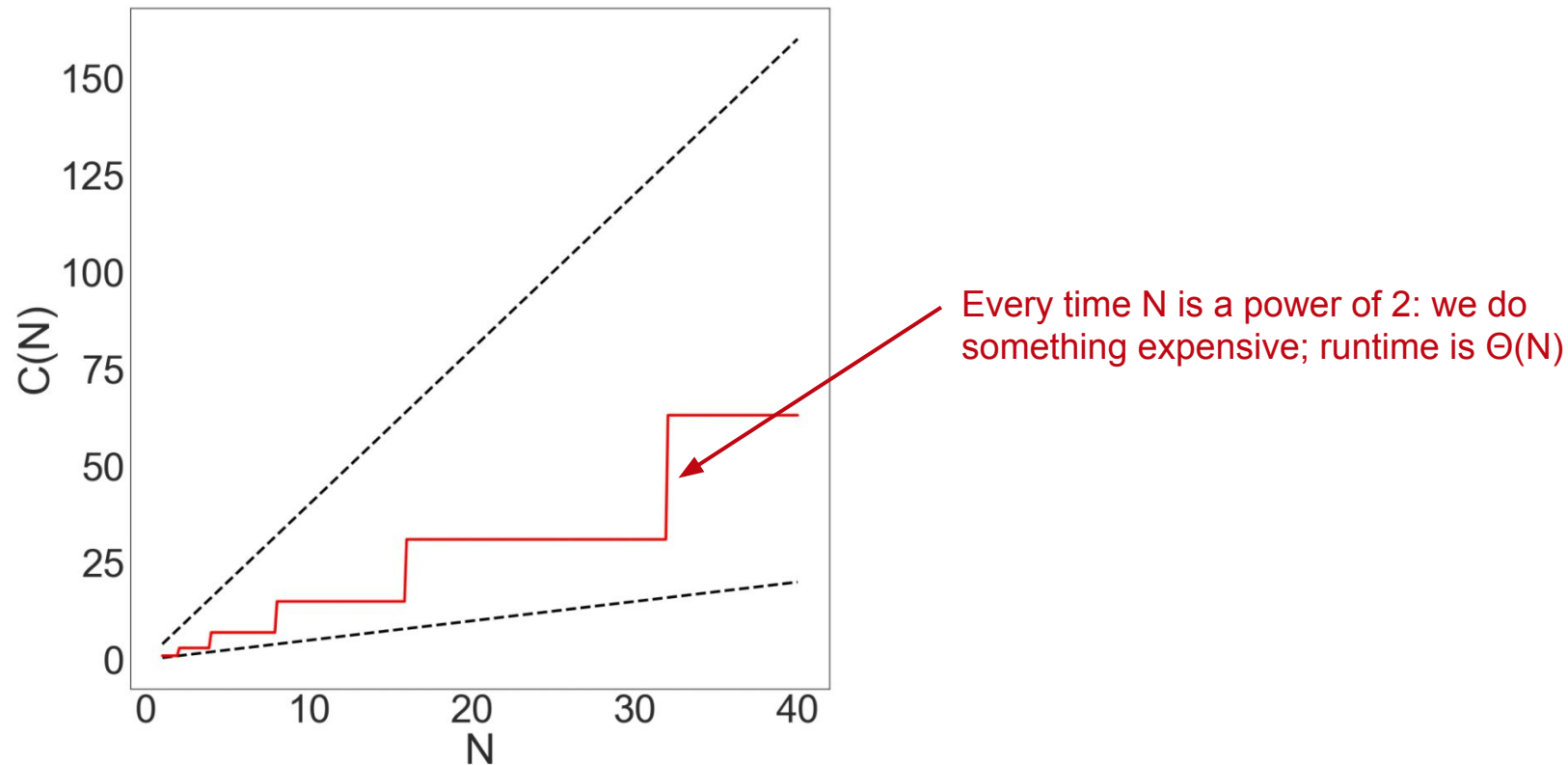
Solution: C(N) is Linear

Or looking at a table, we see that for large N, C(N) is always between 0.5N and 4N.

N	C(N)	0.5 N (below)	4N (above)
1	1	0.5	8
4	$1 + 2 + 4 = 7$	2	16
7	$1 + 2 + 4 = 7$	3.5	28
8	$1 + 2 + 4 + 8 = 15$	4	32
27	$1 + 2 + 4 + 8 + 16 = 31$	13.5	108
185	$\dots + 64 + 128 = \mathbf{255}$	92.5	740
715	$\dots + 256 + 512 = \mathbf{1023}$	357.5	2,860

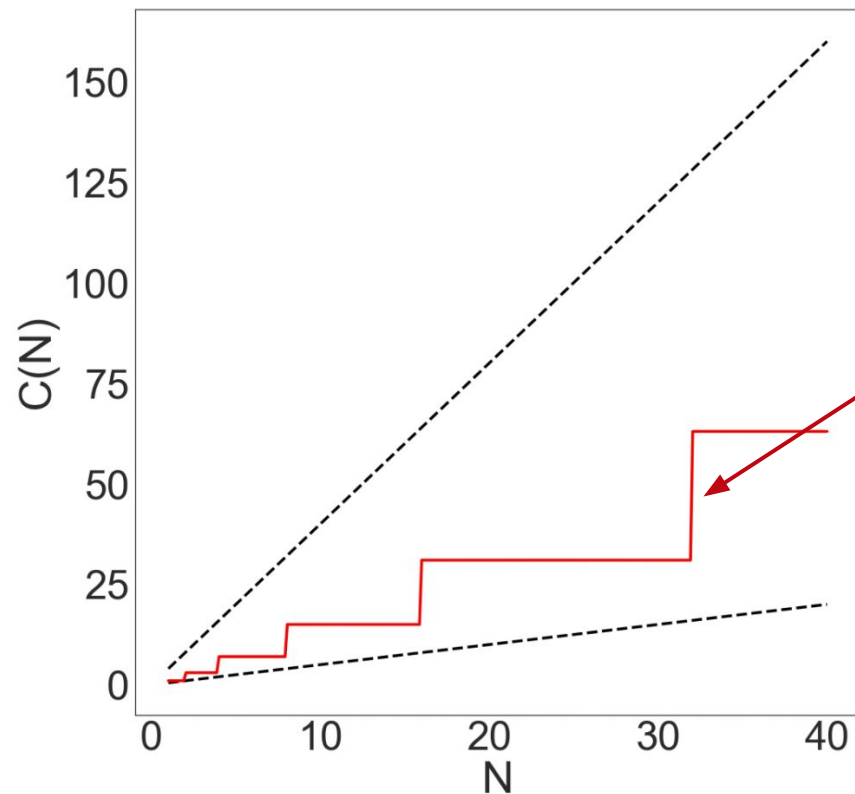
We saw that printParty is $\Theta(N)$.

- It turns out we've seen code that follows this pattern before



We saw that printParty is $\Theta(N)$.

- It turns out we've seen code that follows this pattern before



Every time N is a power of 2: we do something expensive; runtime is $\Theta(N)$

ArrayList resizing!

There is No Magic Shortcut for Asymptotic Analysis

Lecture 15, CS61B, Spring 2026

More Asymptotic Notation

- Asymptotic Review
- Big Omega

Disjoint Sets Review

Asymptotic Analysis Examples

- Nested For Loops
- A Trickier Nested For Loop
- **There is No Magic Shortcut for Asymptotic Analysis**
- More Loop Examples

There is no magic shortcut for asymptotic analysis problems (well... [usually](#))

- Runtime analysis often requires careful thought.
- CS70 and especially CS170 will cover this in much more detail.
- This is not a math class, though we'll expect you to know these:
 - $1 + 2 + 3 + \dots + Q = Q(Q+1)/2 = \Theta(Q^2)$ ← Sum of First Natural Numbers ([Link](#))
 - $1 + 2 + 4 + 8 + \dots + Q = 2Q - 1 = \Theta(Q)$ ← Sum of First Powers of 2 ([Link](#))

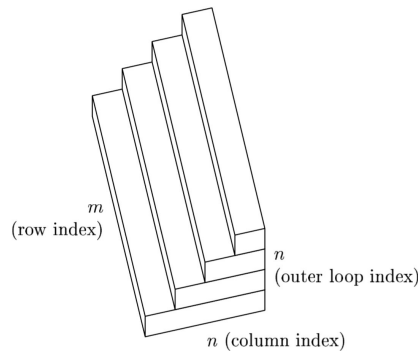
Where Q is a power of 2.

```
public static void printParty(int N) {  
    for (int i = 1; i <= N; i = i * 2) {  
        for (int j = 0; j < i; j += 1) {  
            System.out.println("hello");  
            int ZUG = 1 + 1;  
        }  
    }  
}
```

There is no magic shortcut for asymptotic analysis problems (well... [usually](#))

- Runtime analysis often requires careful thought.
- CS70 and especially CS170 will cover this in much more detail.
- This is not a math class, though we'll expect you to know these:
 - $1 + 2 + 3 + \dots + Q = Q(Q+1)/2 = \Theta(Q^2)$ ← Sum of First Natural Numbers ([Link](#))
 - $1 + 2 + 4 + 8 + \dots + Q = 2Q - 1 = \Theta(Q)$ ← Sum of First Powers of 2 ([Link](#))
- Strategies:
 - Find exact sum.
 - Write out examples.
 - Draw pictures.

QR decomposition runtime,
from “Numerical Linear
Algebra” by Trefethen.



The $m \times n$ rectangle at the bottom corresponds to the first pass through the outer loop, the $m \times (n - 1)$ rectangle above it to the second pass, and so on.

More Loop Examples

Lecture 15, CS61B, Spring 2026

More Asymptotic Notation

- Asymptotic Review
- Big Omega

Disjoint Sets Review

Asymptotic Analysis Examples

- Nested For Loops
- A Trickier Nested For Loop
- There is No Magic Shortcut for Asymptotic Analysis
- **More Loop Examples**

More Loop Examples

```
public static void printParty(int N) {  
    for (int i = 1; i <= N; i = i * 2) {  
        for (int j = 0; j < i; j += 1) {  
            System.out.println("hello");  
            int ZUG = 1 + 1;  
        }  
    }  
}
```

New order of growth	Modification to printParty to have this order of growth?
1	
$\log N$	
N	(none)
$N \log N$	
N^2	

More Loop Examples: Order of growth 1

```
public static void printParty(int N) {  
    for (int i = 1; i <= N; i = i * 2) {  
        for (int j = 0; j < i; j += 1) {  
            System.out.println("hello");  
            int ZUG = 1 + 1;  
        }  
    }  
}
```

return;

New order of growth	Modification to printParty to have this order of growth?
1	Replace inner loop with return;
$\log N$	
N	(none)
$N \log N$	
N^2	

More Loop Examples

```
public static void printParty(int N) {  
    for (int i = 1; i <= N; i = i * 2) {  
        for (int j = 0; j < i; j += 1) {  
            System.out.println("hello");  
            int ZUG = 1 + 1;  
        }  
    }  
}
```

New order of growth	Modification to printParty to have this order of growth?
1	Replace inner loop with return;
$\log N$	
N	(none)
$N \log N$	
N^2	

More Loop Examples: Order of growth 1

```
public static void printParty(int N) {  
    for (int i = 1; i <= N; i = i * 2) {  
        for (int j = 0; j < i; j += 1) {  
            System.out.println("hello");  
            int ZUG = 1 + 1;  
        }  
    }  
}
```

```
System.out.println("hello");
```

New order of growth	Modification to printParty to have this order of growth?
1	Replace inner loop with return;
$\log N$	Replace inner loop with constant-time operation
N	(none)
$N \log N$	
N^2	

More Loop Examples: Order of growth 1

```
public static void printParty(int N) {  
    for (int i = 1; i <= N; i = i * 2) {  
        for (int j = 0; j < N; j += 1) {  
            System.out.println("hello");  
            int ZUG = 1 + 1;  
        }  
    }  
}
```



`System.out.println("hello");`

New order of growth	Modification to printParty to have this order of growth?
1	Replace inner loop with return;
log N	Replace inner loop with constant-time operation
N	(none)
N log N	Inner loop always runs N times
N ²	

More Loop Examples: Order of growth 1

```
public static void printParty(int N) {  
    for (int i = 1; i <= N; i = i * 2) { i = i + 1  
        for (int j = 0; j < i; j += 1) {  
            System.out.println("hello");  
            int ZUG = 1 + 1; }  
    }  
}
```

New order of growth	Modification to printParty to have this order of growth?
1	Replace inner loop with return;
$\log N$	Replace inner loop with constant-time operation
N	(none)
$N \log N$	Inner loop always runs N times
N^2	i increments

There is no magic shortcut for asymptotic analysis problems (well... [usually](#))

- Runtime analysis often requires careful thought.
- You need to know two sums:
 - $1 + 2 + 3 + \dots + Q = Q(Q+1)/2 = \Theta(Q^2)$ ← Sum of First Natural Numbers ([Link](#))
 - $1 + 2 + 4 + 8 + \dots + Q = 2Q - 1 = \Theta(Q)$ ← Sum of First Powers of 2 ([Link](#))
- Strategies:
 - Find exact sum.
 - Write out examples.
 - Draw pictures.